

TPE 3 — Parcours en Profondeur (DFS)

Applications avancées : Cycles, Tri topologique, Timestamps

Durée: 2-3h
Travail: Individuel/Binôme
Rendu: dfs.cpp + capture
Deadline: Début Séance 4

■ Objectif

Implémenter l'algorithme de **Parcours en Profondeur (DFS)** avec ses applications avancées. Le repository GitHub contient tout le code de base, les tests, et la documentation complète. Votre tâche : compléter le fichier **session3/tpe/src/dfs.cpp** avec les 6 fonctions demandées. **Consultez les slides de la Séance 3 (pages 13-40)** pour revoir l'algorithme DFS et ses applications en détail.

- **Repository GitHub** : <https://github.com/shumpaga/graph-theory-course>
- **Dossier TPE3** : session3/tpe/
- **Documentation complète** : README.md, docs/ENONCE.md, docs/BAREME.md

■ Arborescence du projet

```
graph-theory-course/  
  ■■■ session3/tpe/  
    ■■■ README.md # Instructions complètes  
    ■■■ Makefile # Compilation (make, make test)  
    ■■■ src/  
      ■ ■■■ graph.h/cpp # ✓ Classe Graph (fournie)  
      ■ ■■■ stack.h/cpp # ✓ Classe Stack (fournie)  
      ■ ■■■ dfs.h # ✓ Signatures (fourni)  
      ■ ■■■ dfs.cpp # ■■■ À COMPLÉTER (6 fonctions)  
      ■ ■■■ main.cpp # ✓ Programme interactif  
    ■■■ tests/  
      ■ ■■■ test_dfs.cpp # ✓ 26 tests automatiques  
    ■■■ data/  
      ■ ■■■ graph_simple.txt # 6 sommets, 7 arêtes  
      ■ ■■■ graph_dag.txt # DAG pour tri topologique  
      ■ ■■■ graph_cycle.txt # Graphe avec cycle  
      ■ ■■■ graph_courses.txt # Prérequis de cours  
    ■■■ docs/  
      ■■■ ENONCE.md # Spécifications détaillées  
      ■■■ BAREME.md # Barème complet (27 pts)
```

■ Niveaux de validation

Niveau	Points	Fonctions (dans src/dfs.cpp)
BASE	10 pts	dfs_recursive(), dfs_iterative(), dfs_timestamps()

STANDARD	12 pts	has_cycle(), topological_sort()
BONUS	3 pts	find_path_dfs()
CODE	+2 pts	Code très bien commenté
TOTAL	27 pts	Note ramenée sur 25

■ Rappel DFS (voir Séance 3, slides 13-20)

Principe : Explorer le graphe **en profondeur** avec une pile LIFO ou récursion. DFS explore le plus loin possible avant de revenir en arrière. **Structures clés** : Stack S (pile), visited[] (sommets visités), discovery[] (temps découverte), finish[] (temps fin), in_stack[] (détection cycles). **Complexité** : $O(V+E)$ temps, $O(V)$ espace.

■ Les 6 fonctions à implémenter

Ouvrez **src/dfs.cpp**. Chaque fonction contient des **TODO** numérotés avec instructions étape par étape. Consultez **docs/ENONCE.md** pour algorithmes détaillés.

#	Fonction	Description	Pts
1	dfs_recursive(g, source)	Parcours récursif simple, affiche ordre de visite	3
2	dfs_iterative(g, source)	Parcours itératif avec pile explicite	3
3	dfs_timestamps(g, discovery[], finish[])	Calcule temps de découverte et fin pour chaque sommet	4
4	has_cycle(g)	Détecte si le graphe contient un cycle	6
5	topological_sort(g)	Retourne tri topologique (NULL si cycle)	6
6	find_path_dfs(g, src, dest, path)	Trouve un chemin entre deux sommets avec DFS	3

■ Ce qui est fourni (NE PAS MODIFIER)

- **graph.h/cpp** : Classe Graph complète avec liste d'adjacence
- **stack.h/cpp** : Classe Stack LIFO (push, pop, top, empty)
- **dfs.h** : Signatures des 6 fonctions DFS
- **main.cpp** : Programme interactif avec menu pour tester
- **tests/test_dfs.cpp** : 26 tests automatiques (10 BASE + 12 STANDARD + 4 BONUS)

■ Exemple de TODO dans dfs.cpp

```
void dfs_recursive(Graph& g, int source) {
    // TODO 1 : Créer vector visited(V, false)
    // TODO 2 : Appeler fonction auxiliaire récursive dfs_visit(source)
}

void dfs_visit(Graph& g, int u, vector& visited) {
    // TODO 3 : Marquer u visité : visited[u] = true
    // TODO 4 : Afficher u
    // TODO 5 : Pour chaque voisin v non visité, appeler dfs_visit(v)
}
```

■ Pièges fréquents (65% des erreurs !)

Ces erreurs sont présentes dans **65% des copies**. Lisez attentivement !

#	Piège	Solution
---	-------	----------

1	Oublier de marquer visited[] dans DFS récursif	Toujours marquer visited[u] = true AVANT de visiter les voisins. Sinon boucle infinie !
2	Marquer visited[] trop tôt dans DFS itératif	Marquer visited[u] = true APRÈS avoir dépilé u, pas avant empiler. Sinon sommets en double.
3	Ne pas initialiser discovery[] et finish[] à -1	Utiliser vector<int> discovery(V, -1), finish(V, -1) pour détecter sommets non visités.
4	Oublier in_stack[] pour détection cycle orienté	Créer in_stack[] séparé de visited[]. in_stack[u] = true pendant exploration, false après.
5	Tri topologique sur graphe avec cycle	TOUJOURS vérifier absence de cycle AVANT tri topologique. Si cycle → retourner NULL.
6	Utiliser std::stack au lieu de Stack fournie	Le TPE impose la classe Stack dans stack.h. Utiliser std::stack = -3 points de pénalité.

■ NOTE IMPORTANTE — Utilisation de la classe Stack

Vous **DEVEZ** utiliser la classe **Stack** fournie dans stack.h/cpp pour la version itérative. L'utilisation de **std::stack** de la STL entraînera une **pénalité de -3 points**. La classe Stack est déjà implémentée, testée, et prête à l'emploi avec les méthodes : push(x), pop(), top(), empty(). Consultez stack.h pour voir les signatures.

■ Format des fichiers graphes (data/)

Format : Ligne 1 = "V E directed", puis lignes "u v" (arêtes).

Exemple (graph_dag.txt) :

```
5 6 1      ← 5 sommets, 6 arêtes, orienté
0 1        ← Arête 0→1
0 2
1 3
2 3
2 4
3 4
```

Graphe résultant : DAG avec sommets {0,1,2,3,4} permettant tri topologique.

■ Graphes de test

Fichier	Contenu	Usage
graph_simple.txt	6 sommets, 7 arêtes, connexe	Tests BASE (fonctions 1-3)
graph_dag.txt	5 sommets, DAG pour tri topo	Tests STANDARD (fonction 5)
graph_cycle.txt	4 sommets, contient cycle	Tests STANDARD (fonction 4)
graph_courses.txt	Prérequis cours GI2	Validation complète (contexte réel)

■ Compilation et tests

```
# Compilation et tests automatiques
make clean
make test
./test_dfs

# Programme interactif (menu avec 6 fonctions)
make
./main data/graph_simple.txt

# Aide
make help
```

■ Sortie attendue des tests (26 tests)

1. Consultez **README.md** — Documentation complète du projet
2. Lisez **docs/ENONCE.md** — Algorithmes détaillés pour chaque fonction
3. Relisez les **slides** — Séance 3 pages 13-20 (DFS récursif/itératif) et 26-40 (timestamps, applications)
4. Procédez par niveau — BASE (10 tests) → STANDARD (12 tests) → BONUS (4 tests)

5. **Testez fréquemment** — make test après chaque TODO complété
6. **Évitez les pièges** — Relire la liste des 6 pièges fréquents (page 3)
7. **Programme interactif** — Utilisez ./main pour tester manuellement
8. **Timestamps et cycles** — Bien comprendre in_stack[] pour détection cycle orienté

Bon courage ! Le code de base est sur GitHub. Les TODO dans src/dfs.cpp vous guident étape par étape. DFS et ses applications sont fondamentaux en théorie des graphes — prenez le temps de bien comprendre !

